رواد مصر الرقمية

وزارة الاتصـــــالات
وتكنولوجيا المعلومات

Machine Learning

Session 4

Agenda:

1	Decision Trees
2	Naive Bayes

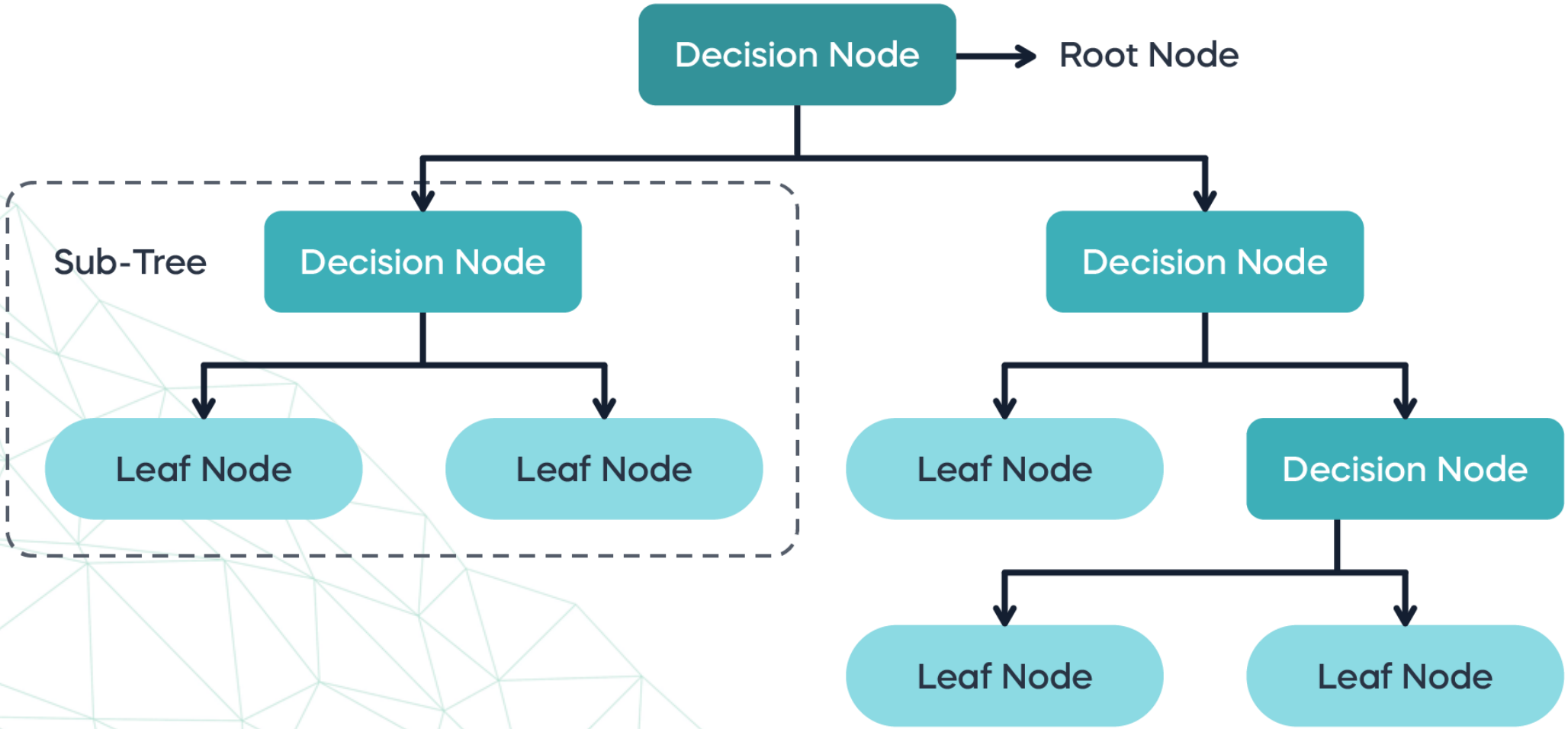
Decision Tree

Decision Tree

A **Decision Tree** is a supervised machine learning algorithm used for both classification and regression tasks. The key idea is to split the dataset into smaller subsets based on feature values, and this process continues until a stopping condition is met (for example, no further splitting is possible, or maximum depth is reached). The result is a tree-like structure where :

- **Nodes** represent tests on an attribute (feature).
- **Branches** represent the outcome of those tests.
- **Leaves** represent class labels (for classification tasks) or continuous values (for regression tasks).

Decision Tree



Decision Tree

Key Concepts

- **Root Node**: The topmost node in a tree, representing the initial decision.
- **Leaf Nodes**: The terminal nodes that hold the output class labels or values.
- **Splitting**: The process of dividing nodes into sub-nodes based on a feature.
- **Pruning**: Reducing the size of the tree to prevent overfitting by trimming branches that provide little predictive power.

Decision Tree

How to Determine the Best Split?

The "best split" in a decision tree is determined by using a **node impurity measure**—an indicator of how well the data at a node is classified or how "pure" a node is.

Decision Tree

Gini Index (Gini Impurity)

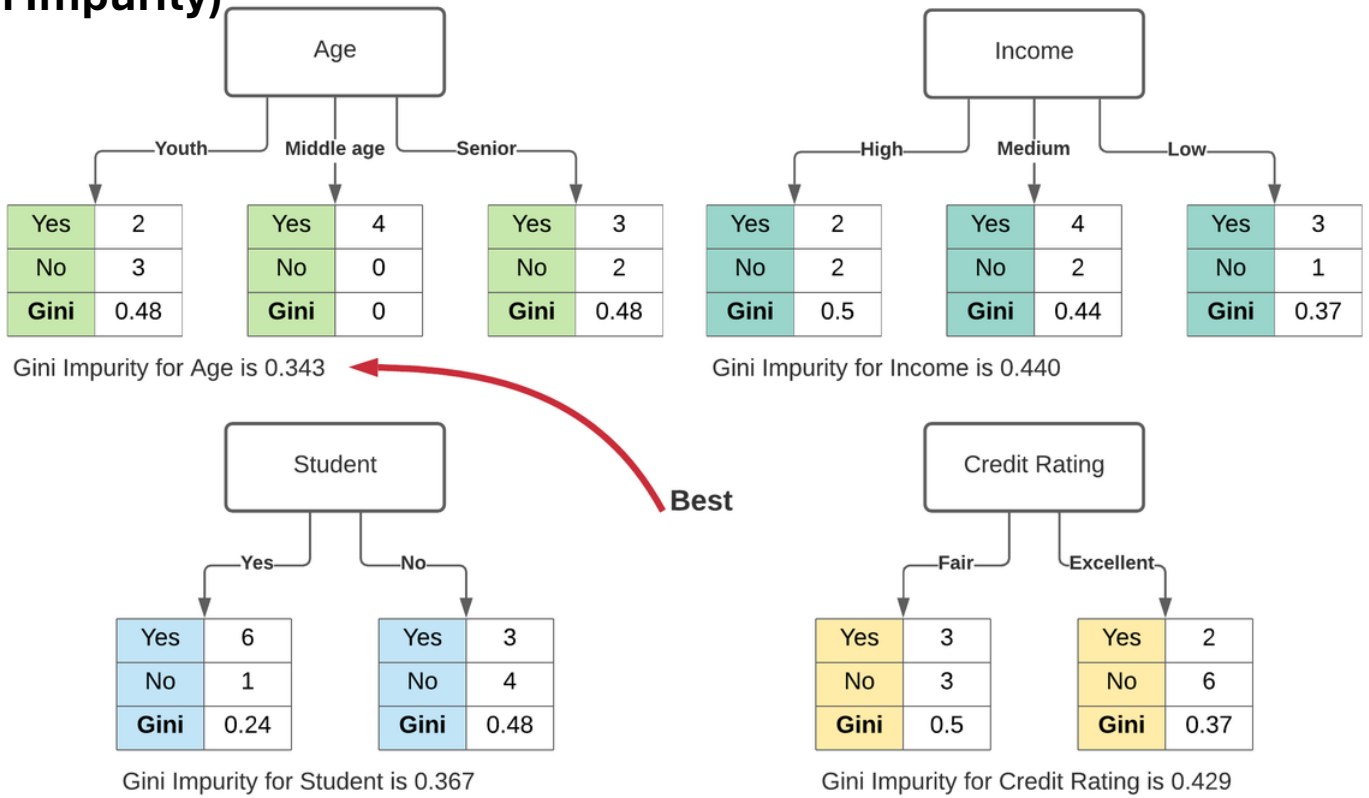
The **Gini Index** measures the probability that a randomly chosen sample would be misclassified if it were randomly assigned a label based on the distribution of class labels at the node.

Formula: $Gini(p) = 1 - \sum_{i=1}^{n} p_i^2$

Where **p** is the probability of an instance belonging to class **i**

Decision Tree

Gini Index (Gini Impurity)



Decision Tree

Gini Index (Gini Impurity)

Example

Let's consider a node that contains 100 samples. These samples belong to two classes: **Class A** and **Class B**. The node contains:

- 70 samples of Class A
- 30 samples of Class B

Decision Tree

Gini Index (Gini Impurity)

Example

1. Calculate the probabilities

   - Probability of Class A (p_A): $70/100 = 0.7$
   - Probability of Class B (p_B): $30/100 = 0.3$

2. Calculate the Gini Index

$$Gini = 1 - (0.7^2 + 0.3^2) = 1 - (0.49 + 0.09) = 1 - 0.58 = 0.42$$

The Gini Index of this node is 0.42, indicating some impurity (a value of 0 would mean the node is perfectly pure).

Decision Tree

Gini Index (Gini Impurity)

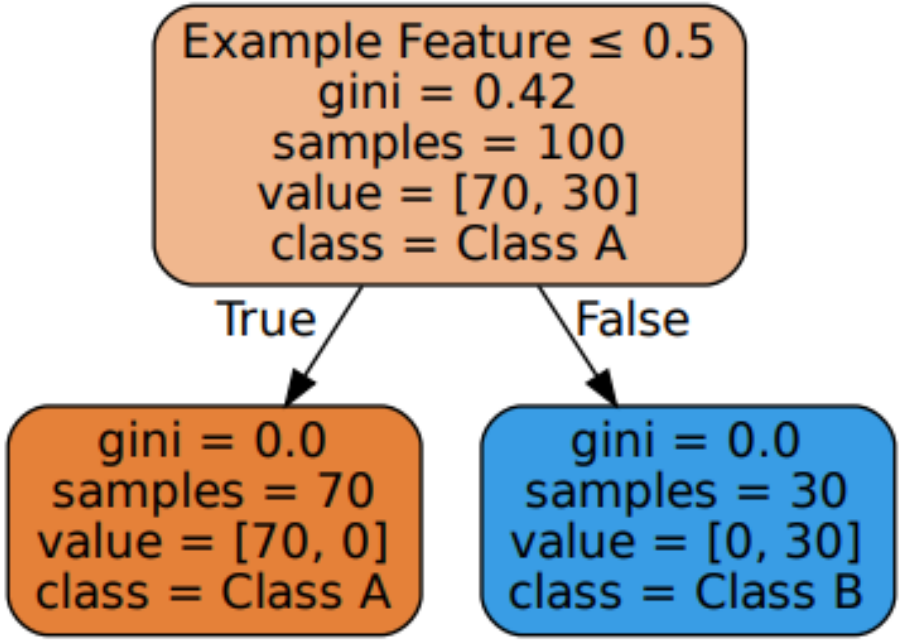
Example

High Impurity (Gini ≈ 0.5):

Class A	Class B
50%	50%

Low Impurity (Gini ≈ 0.0):

Class A	Class B
90%	10%



Decision Tree

Entropy (Information Gain)

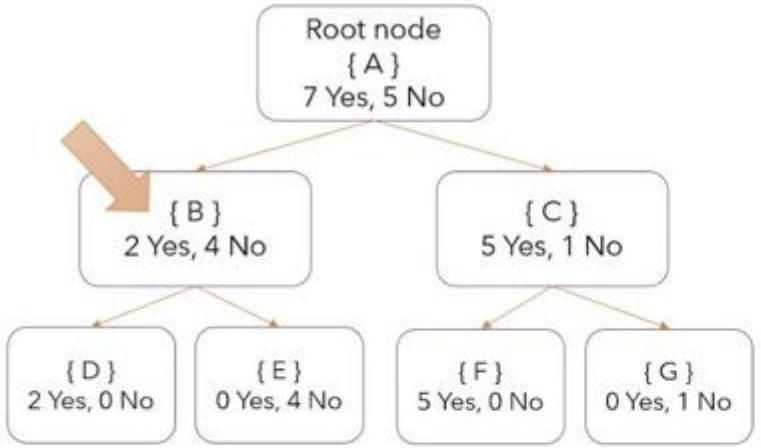
Entropy measures the uncertainty or disorder in a node. The more mixed the classes are at a node, the higher the entropy.

Formula: $Entropy(p) = -\sum_{i=1}^{n} p_i \log_2(p_i)$

Where pi is the probability of an instance belonging to class i

Decision Tree

Entropy (Information Gain)



Entropy

$$Entropy(S) = \sum_{i=1}^{c} -pi\ log_2(pi)$$

$p(Yes) = \frac{P(Yes)}{P(Yes)+P(No)}$, for "Yes" component of node A

$$Entropy(B) = -\frac{2}{2+4}\ log_2\left(\frac{2}{2+4}\right) - \frac{4}{4+2}\ log_2\left(\frac{4}{4+2}\right)$$

$$= -\frac{2}{6}\ log_2\left(\frac{2}{6}\right) - \frac{4}{6}\ log_2\left(\frac{4}{6}\right)$$

$$= 0.92\ bits$$

$$Entropy(C) = -\frac{5}{6}\ log_2\left(\frac{5}{6}\right)\ -\frac{1}{6}\ log_2\left(\frac{1}{6}\right)$$

$$= 0.65 \text{ bits}$$

Entropy ranges from 0 to 1, where 0 consider as pure sub − tree & 1 as most impure sub − tree

Decision Tree

Entropy (Information Gain)

Example

Let’s use the same node as above with 100 samples

- 70 samples of Class A
- 30 samples of Class B

Decision Tree

Entropy (Information Gain)

Example

1. Calculate the probabilities

   - Probability of Class A (p_A): $70/100 = 0.7$
   - Probability of Class B (p_B): $30/100 = 0.3$

2. Calculate the Entropy

$$Entropy = -(0.7\log_2(0.7) + 0.3\log_2(0.3)) = -(0.7 \times -0.5146 + 0.3 \times -1.736) = 0.88$$

The entropy of this node is **0.88**, which is closer to 1, indicating higher impurity.

Decision Tree

Entropy (Information Gain)

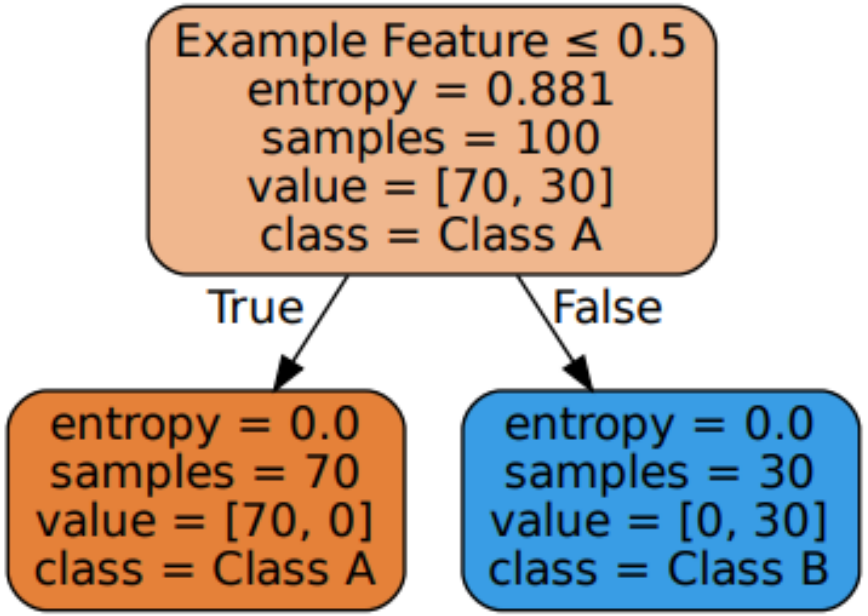
Example

High Entropy (≈ 1)

Class A	Class B
50%	50%

Low Entropy (≈ 0)

Class A	Class B
100%	0%



Decision Tree

Misclassification Error

The **Misclassification Error** is the fraction of samples that do not belong to the most common class at a node. It gives a simple estimate of impurity but doesn't provide as much detail as Gini or Entropy.

Formula: $Error(p) = 1 - \max(p_i)$

Where pi is the probability of the most frequent class.

Decision Tree

Misclassification Error

Example

In the same node with 100 samples

- 70 samples of Class A
- 30 samples of Class B

Decision Tree

Misclassification Error

Example

1. Calculate the probabilities
    - Probability of Class A (p_A): $70/100 = 0.7$
2. Calculate the Misclassification Error

$$Error = 1 - 0.7 = 0.3$$

The Misclassification Error is 0.3, indicating that 30% of the samples are misclassified if we predict only the majority class (Class A).

Decision Tree

Misclassification Error

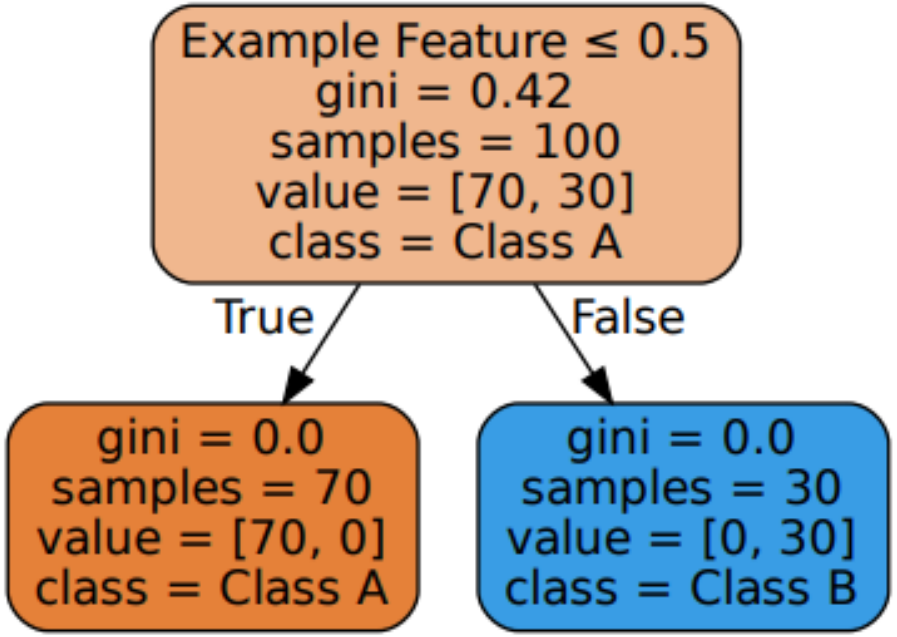
Example

High Misclassification Error (≈ 0.5)

Class A	Class B
50%	50%

High Misclassification Error (≈ 0.5)

Class A	Class B
90%	10%



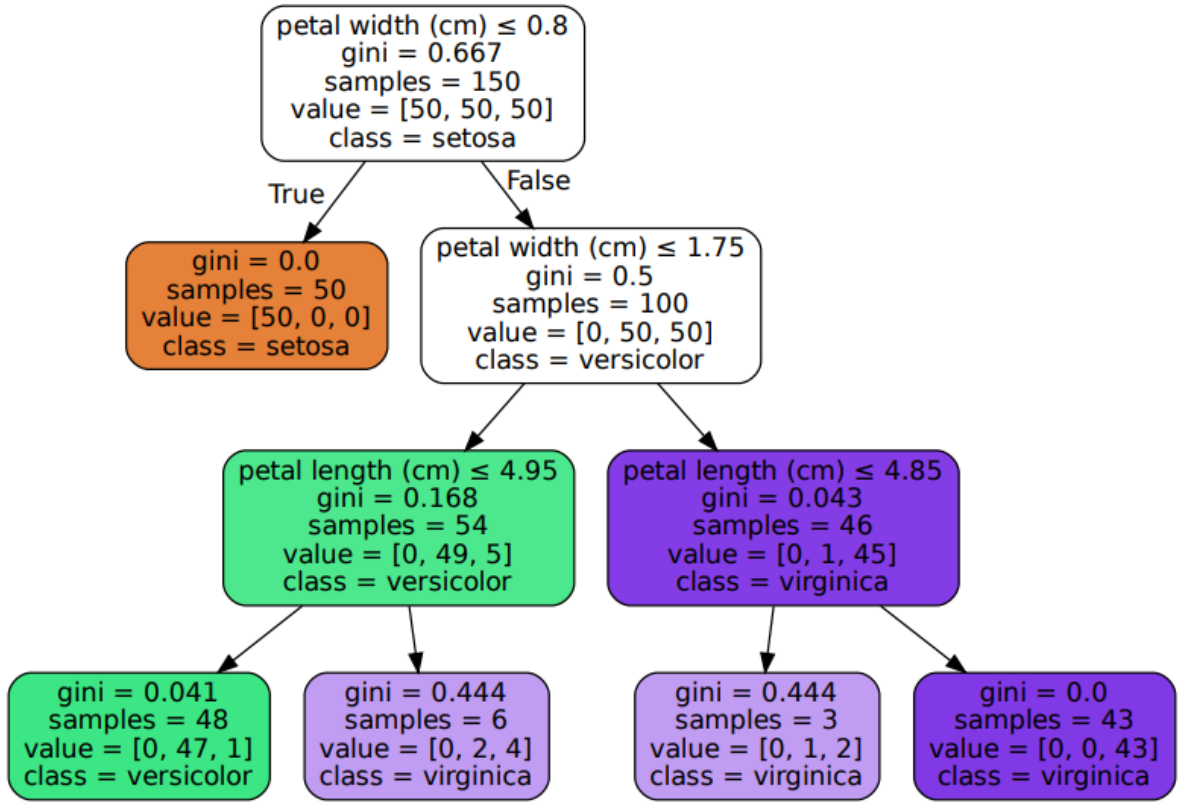
Decision Tree

Summary of Measures of Node Impurity

Measure	Formula	Range	Meaning
Gini Index	$Gini = 1 - \sum p_i^2$	[0, 0.5]	Measures the probability of misclassification.
Entropy	$Entropy = -\sum p_i \log_2(p_i)$	[0, 1]	Measures the level of uncertainty.
Misclassification Error	$Error = 1 - \max(p_i)$	[0, 0.5]	Proportion of samples that are misclassified.

Decision Tree

Decision Tree Analysis of Iris Flower Classification Based on Petal Features



Decision Tree

Decision Tree Analysis of Iris Flower Classification Based on Petal Features

1. Root Node

- Petal width (cm) ≤ 0.8
  - Gini = 0.667 (indicating impurity).
  - 150 total samples, equally distributed across three classes: Setosa, Versicolor, and Virginica.

Decision Tree

Decision Tree Analysis of Iris Flower Classification Based on Petal Features

2. Left Branch (True)

- Gini = 0 (pure node, all samples belong to Setosa).
- 50 samples, all Setosa.

Decision Tree

Decision Tree Analysis of Iris Flower Classification Based on Petal Features

3. Right Branch (False)

- Petal width (cm) ≤ 1.75
    - Gini = 0.5 (impurity, equally split between Versicolor and Virginica).
    - 100 samples (50 Versicolor, 50 Virginica).

Decision Tree

Decision Tree Analysis of Iris Flower Classification Based on Petal Features

4. Further Splits

- Left of the split (Petal length ≤ 4.95)
  - Gini = 0.168 (mostly Versicolor).
  - 54 samples (49 Versicolor, 5 Virginica).
- Right of the split (Petal length ≤ 4.85)
  - Gini = 0.043 (almost pure Virginica).
  - 46 samples (45 Virginica, 1 Versicolor).

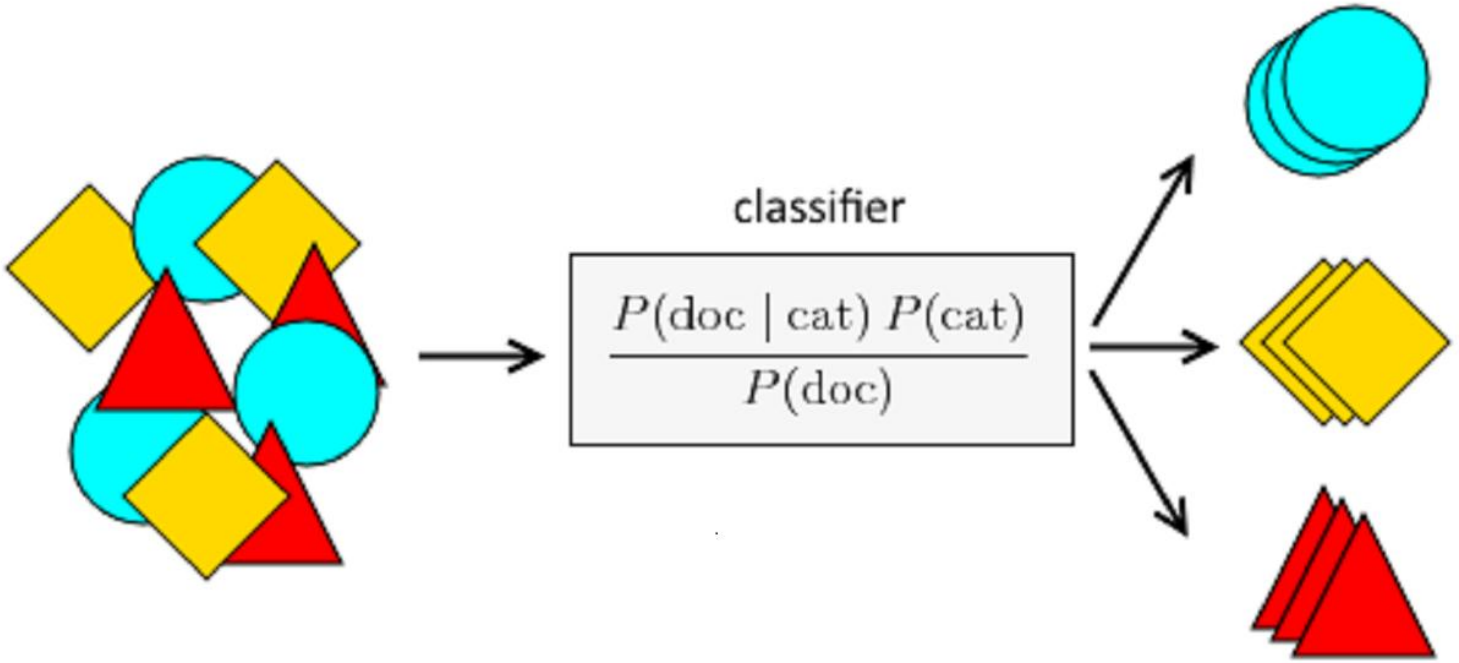
5. Leaf Nodes

- End of the branches where Gini = 0 indicates pure nodes, with samples belonging entirely to one class.

Naive Bayes

Naive Bayes

Naive Bayes is a probabilistic machine learning algorithm based on **Bayes' Theorem**. It's called "naive" because it assumes that all features are **independent** of each other, which is rarely true in real-world scenarios, but it simplifies the computation.



Naive Bayes

Bayes' Theorem gives the probability of a class given a set of features. Mathematically

$$P(C|X) = \frac{P(X|C) \cdot P(C)}{P(X)}$$

Where:

- P($C|X$) is the posterior probability of class C given the feature set X,
- P($X|C$) is the likelihood of feature set X given class C,
- P(C) is the prior probability of class ,
- P(X) is the evidence or overall probability of the feature set X ,

Naive Bayes

Types of Naive Bayes Classifiers

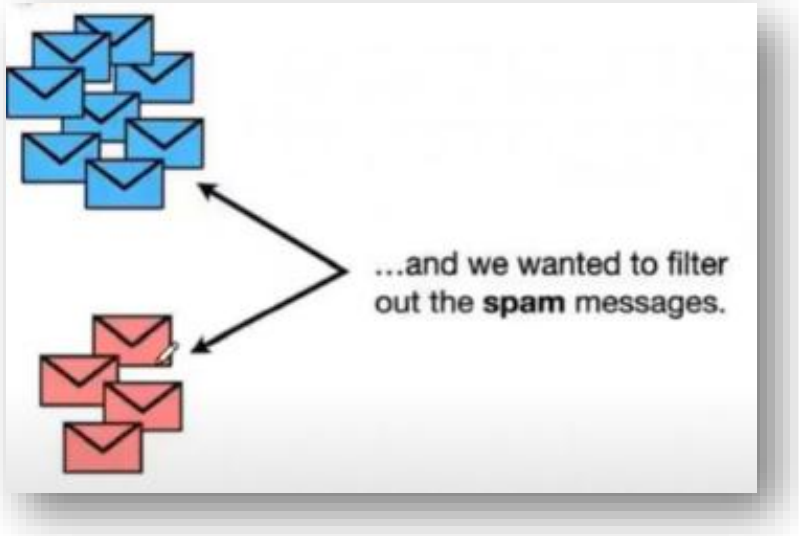
1. **Gaussian Naive Bayes**: Assumes that the continuous features follow a normal (Gaussian) distribution.
2. **Multinomial Naive Bayes**: Used for discrete counts, such as in text classification.
3. **Bernoulli Naive Bayes**: Suitable for binary/boolean features.

Naive Bayes

Spam and Not Spam Classification Using Multinomial Naive Bayes

we need to classify messages into **spam** and **not spam (ham)** categories. To begin, we need to **gather the messages** and **separate them into two groups**: spam and ham.

- **Goal**: Classify whether a message is spam or not.
- **Approach**: We will use a Multinomial Naive Bayes classifier to work on the word frequencies in these messages.

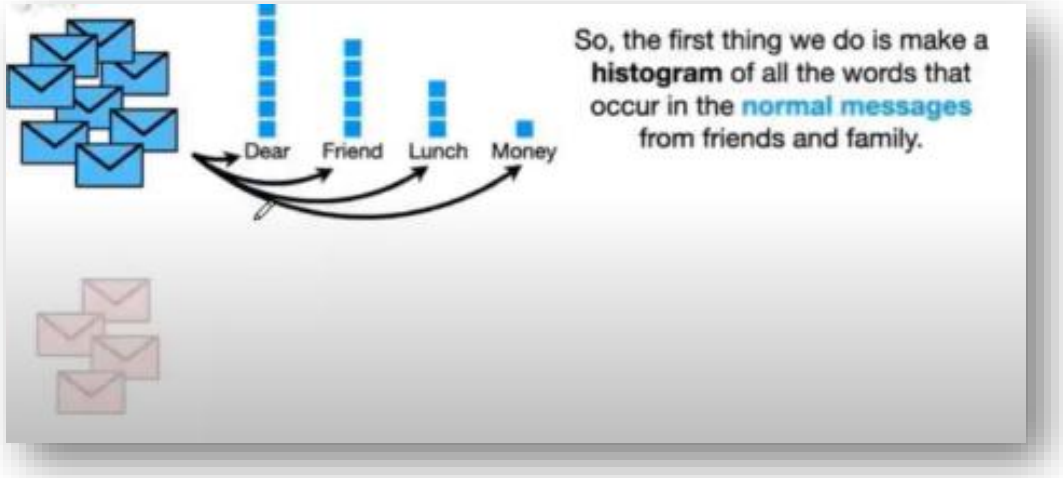


Naive Bayes

Create Word Histograms for Ham Messages

we need to **create histograms** of word frequencies in the ham (non-spam) messages. This step involves counting how often each word occurs in the **ham** messages. For example:

- Words like "Dear," "Friend," "Lunch," and "Money" are counted to understand their frequency in non-spam messages.



Naive Bayes

Calculate Likelihood for Words in Ham (Normal) Messages

calculating the **likelihood** of each word occurring in a **normal (ham) message**. This step is crucial because it helps the model understand how likely certain words are to appear in normal (non-spam) messages.

1. Given Word Frequencies

- We already have the word frequencies from the histogram we created earlier (Step 2). Now, we calculate the probability of each word given that the message is a normal message

For example

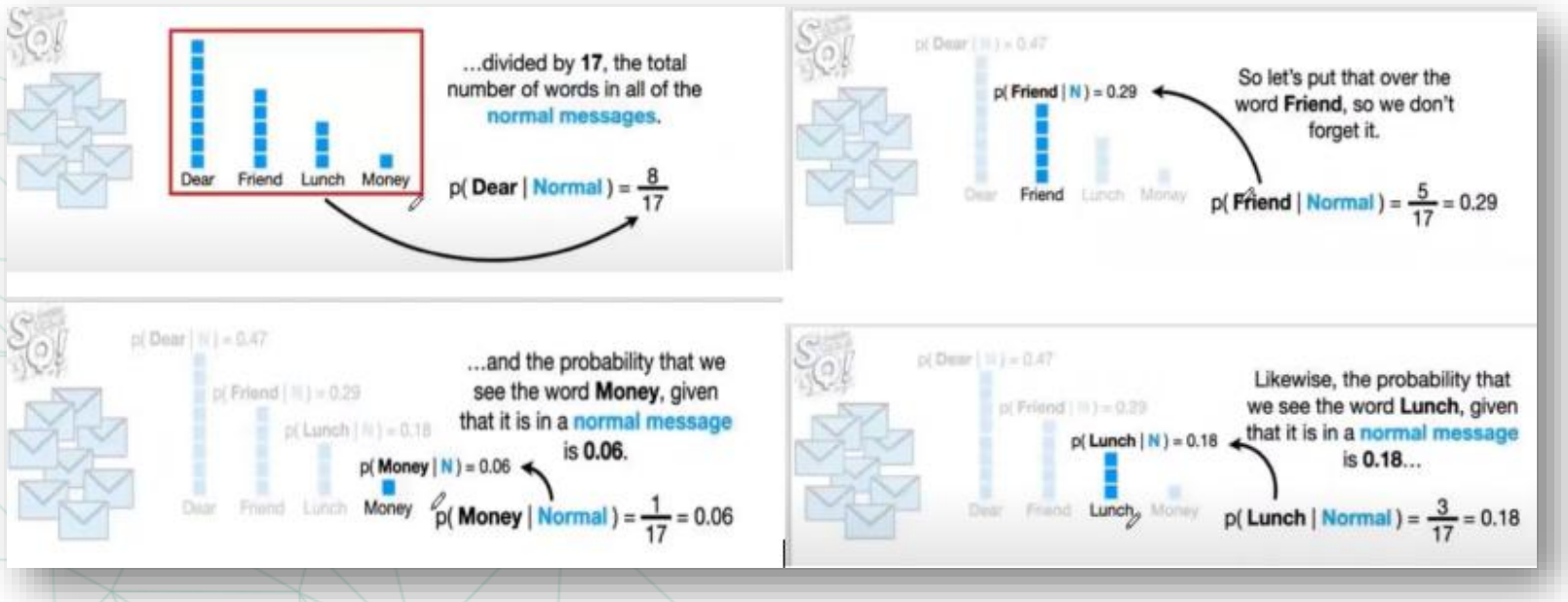
- $P(\text{Dear}|\text{Normal}) = \frac{8}{17} = 0.47$
- $P(\text{Friend}|\text{Normal}) = \frac{5}{17} = 0.29$
- $P(\text{Lunch}|\text{Normal}) = \frac{3}{17} = 0.18$
- $P(\text{Money}|\text{Normal}) = \frac{1}{17} = 0.06$

Naive Bayes

Calculate Likelihood for Words in Ham (Normal) Messages

2. Purpose of Likelihood Calculation

These probabilities help the model learn that certain words, like "Dear" or "Friend," are more likely to appear in normal messages, while words like "Money" are less likely.

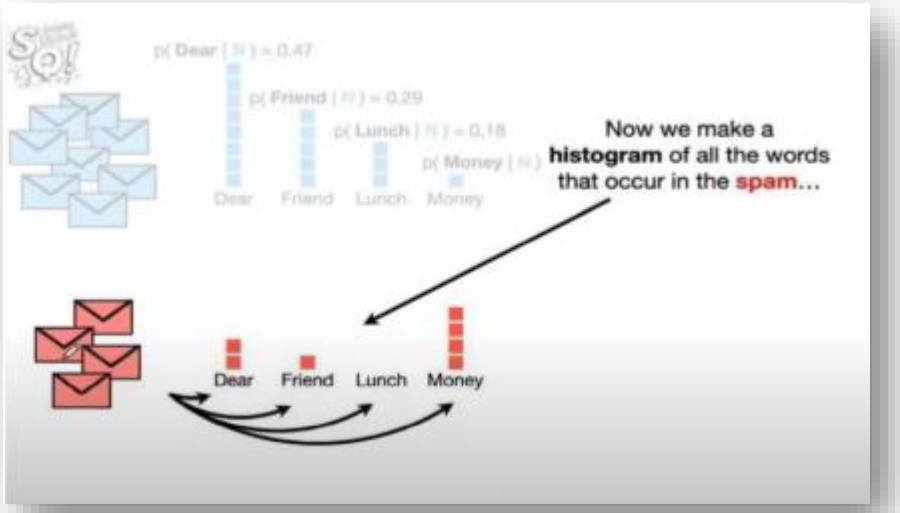


Naive Bayes

Create Word Histogram for Spam Messages

Now, we perform a similar operation for **spam messages**. We need to build a histogram for spam messages to determine how often each word appears in spam messages.

- **Goal**: Identify which words are common in spam messages.
- Similar to the normal message histogram, we will calculate the frequency of each word in spam messages, which will help us estimate the likelihood of a word appearing in spam.



Naive Bayes

Calculate Likelihood for Words in Spam Messages

Now that we have constructed a word histogram for spam messages, we proceed to calculate the **likelihood** of each word appearing in a **spam message**.

1. Word Likelihood in Spam?

- Just like we did for ham messages, we now calculate the likelihood of each word given that the message is spam.

For example

- $P(\text{Dear}|\text{Spam}) = \frac{2}{7} = 0.29$
- $P(\text{Friend}|\text{Spam}) = \frac{1}{7} = 0.14$
- $P(\text{Lunch}|\text{Spam}) = \frac{0}{7} = 0.00$
- $P(\text{Money}|\text{Spam}) = \frac{4}{7} = 0.57$

Naive Bayes

Calculate Likelihood for Words in Spam Messages

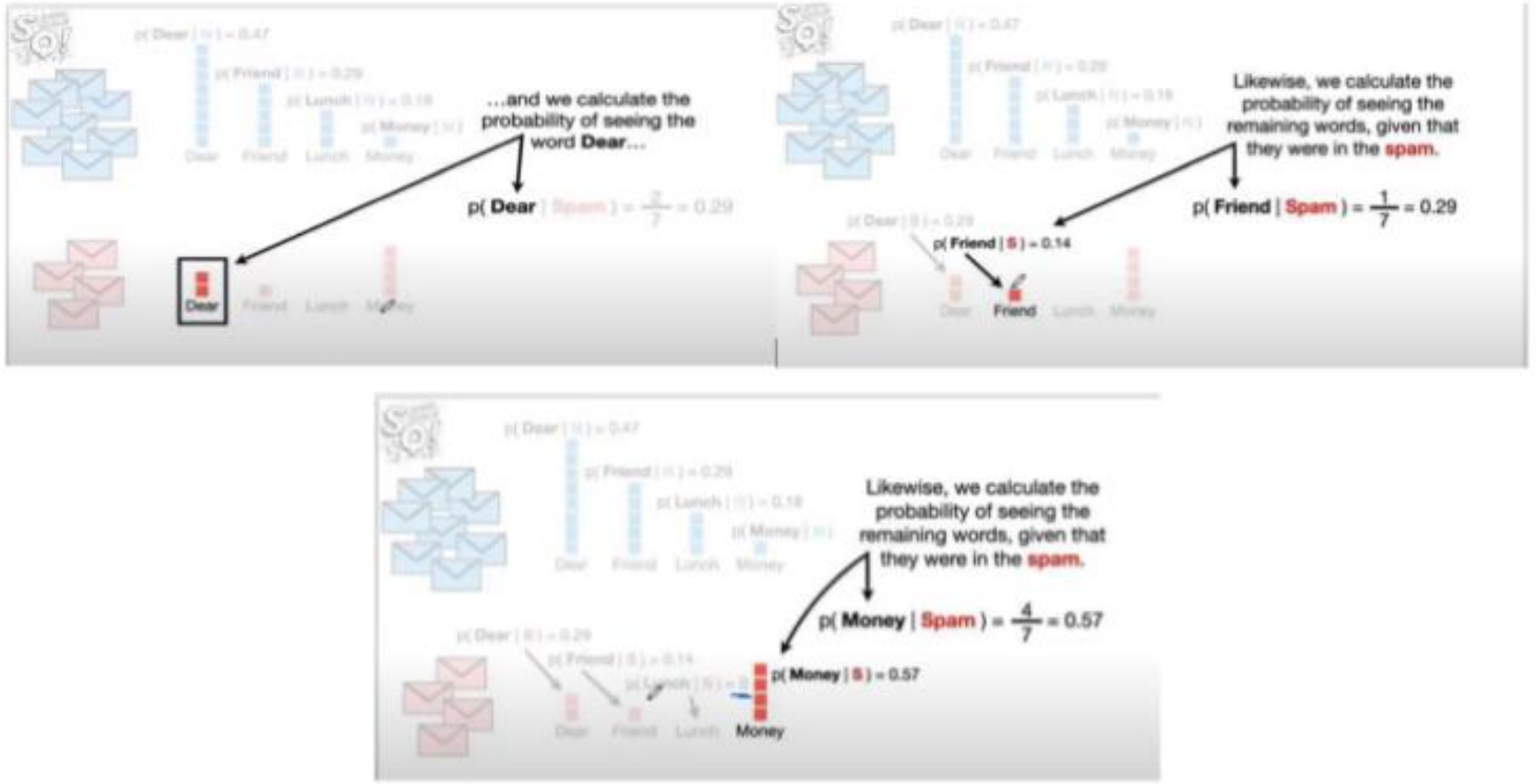
2. Handling Zero Probabilities

- Notice that the word "Lunch" has a probability of 0 because it was not seen in any spam messages. This creates a problem because multiplying by zero would invalidate our model.

- To solve this issue, we use Laplace Smoothing, where we add a small constant (like 1) to every count to avoid zero probabilities. After smoothing, the probabilities are adjusted slightly.

Naive Bayes

Calculate Likelihood for Words in Spam Messages



Naive Bayes

Combine Likelihoods (Conditional Probabilities)

Next, we combine the likelihoods for both normal and spam messages for each word. This is where **conditional probabilities** come into play.

- We now have the probabilities for each word given it is in a normal message **(P(Word|Normal))** and a spam message **(P(Word|Spam)).**

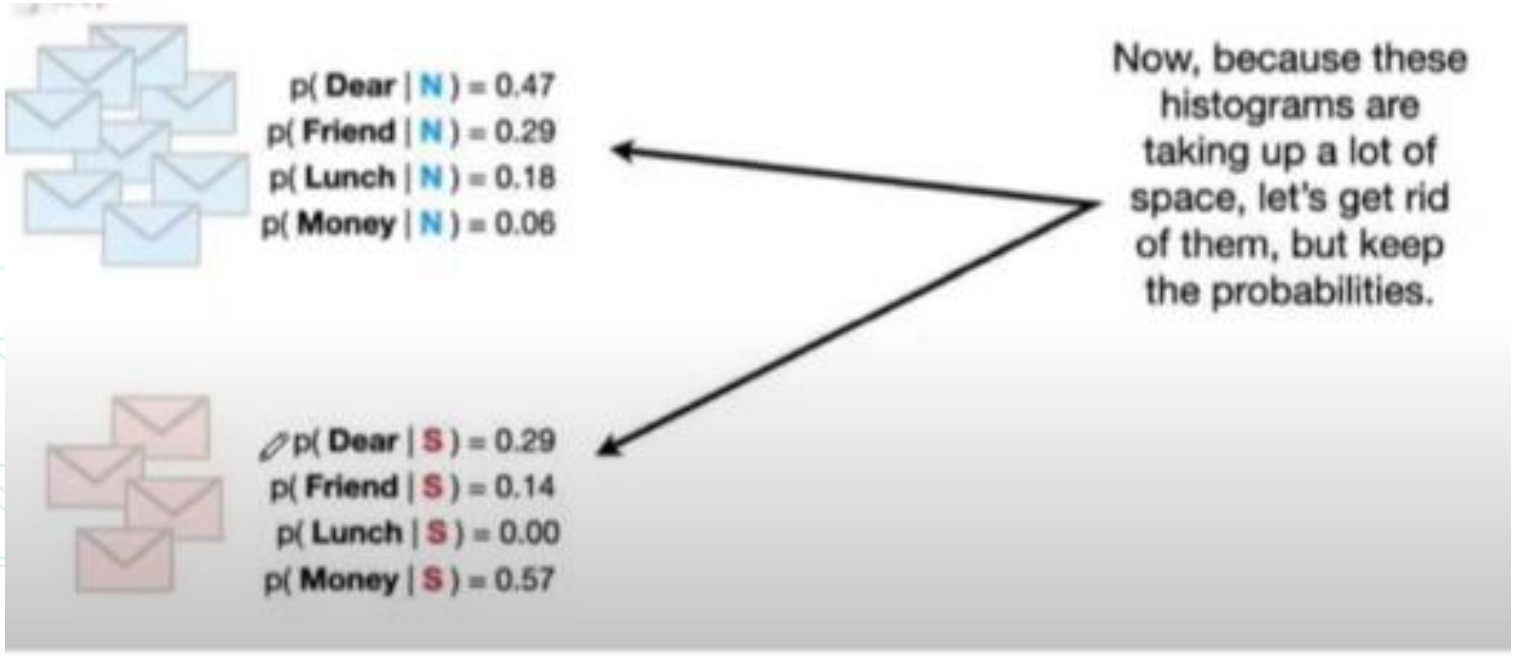
- These probabilities will be used to calculate the overall probability that a message is spam or not, given the presence of these words.

Example

- $P(\text{Dear}|\text{Normal}) = 0.47$ and $P(\text{Dear}|\text{Spam}) = 0.29$
- $P(\text{Money}|\text{Normal}) = 0.06$ and $P(\text{Money}|\text{Spam}) = 0.57$

Naive Bayes

Combine Likelihoods (Conditional Probabilities)



Naive Bayes

Classify New Messages Using Conditional Probabilities

Now that we have calculated the likelihoods for both **spam** and **normal (ham)** messages, we can use these probabilities to classify a new message as either spam or ham.

1. Test the Word "**Dear Friend**":

- We want to classify whether the phrase "Dear Friend" is more likely to appear in a **spam** message or a **normal** message.

Naive Bayes

Classify New Messages Using Conditional Probabilities

2. Prior Probability

- We calculate the **prior probability** of spam and ham based on the proportion of messages in the dataset.

For example

- If 8 out of 12 messages are normal, then $P(\text{Normal}) = \frac{8}{12} = 0.67$.
- Similarly, $P(\text{Spam}) = \frac{4}{12} = 0.33$.

Naive Bayes

Classify New Messages Using Conditional Probabilities

3. Posterior Probability

- Using **Bayes' Theorem**, we multiply the prior probability by the likelihood of seeing the words "Dear Friend" in each class.

- For normal messages:

$$P(\text{Normal}|\text{Dear Friend}) = P(\text{Dear}|\text{Normal}) \times P(\text{Friend}|\text{Normal}) \times P(\text{Normal})$$

  - $P(\text{Normal}|\text{Dear Friend}) = 0.47 \times 0.29 \times 0.67 = 0.09$

- For spam messages:

$$P(\text{Spam}|\text{Dear Friend}) = P(\text{Dear}|\text{Spam}) \times P(\text{Friend}|\text{Spam}) \times P(\text{Spam})$$

  - $P(\text{Spam}|\text{Dear Friend}) = 0.29 \times 0.14 \times 0.33 = 0.01$

Naive Bayes

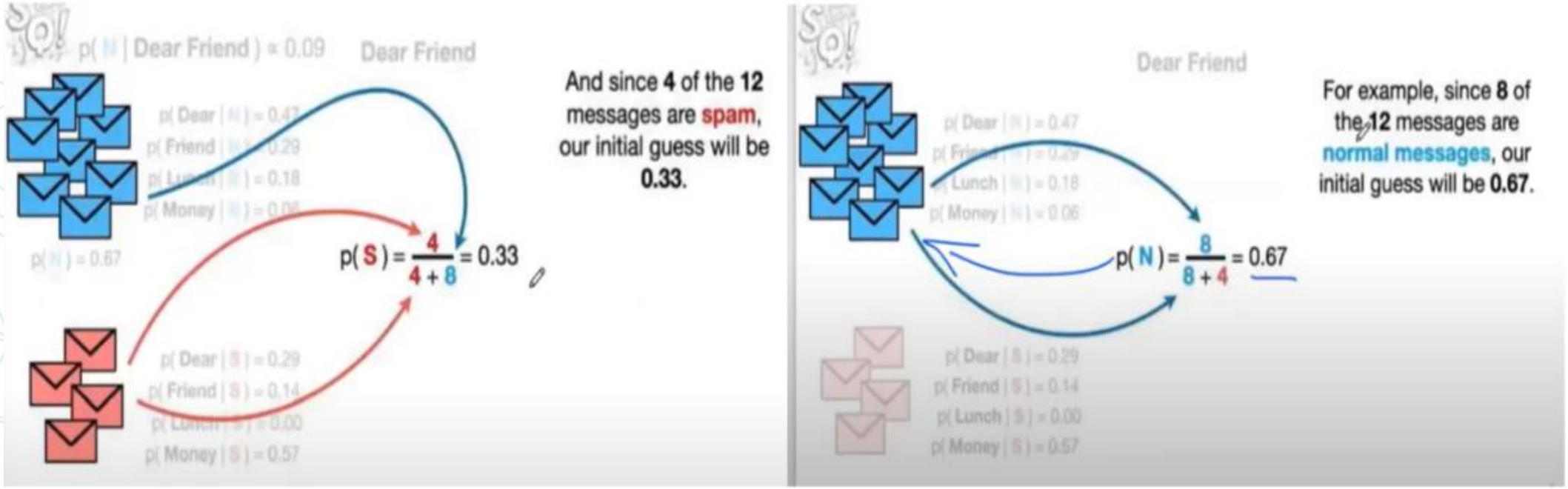
Classify New Messages Using Conditional Probabilities

4. Conclusion

- Since $P(\text{Normal}|\text{Dear Friend}) = 0.09$ is greater than $P(\text{Spam}|\text{Dear Friend}) = 0.01$, we classify the message as **normal (ham)**.

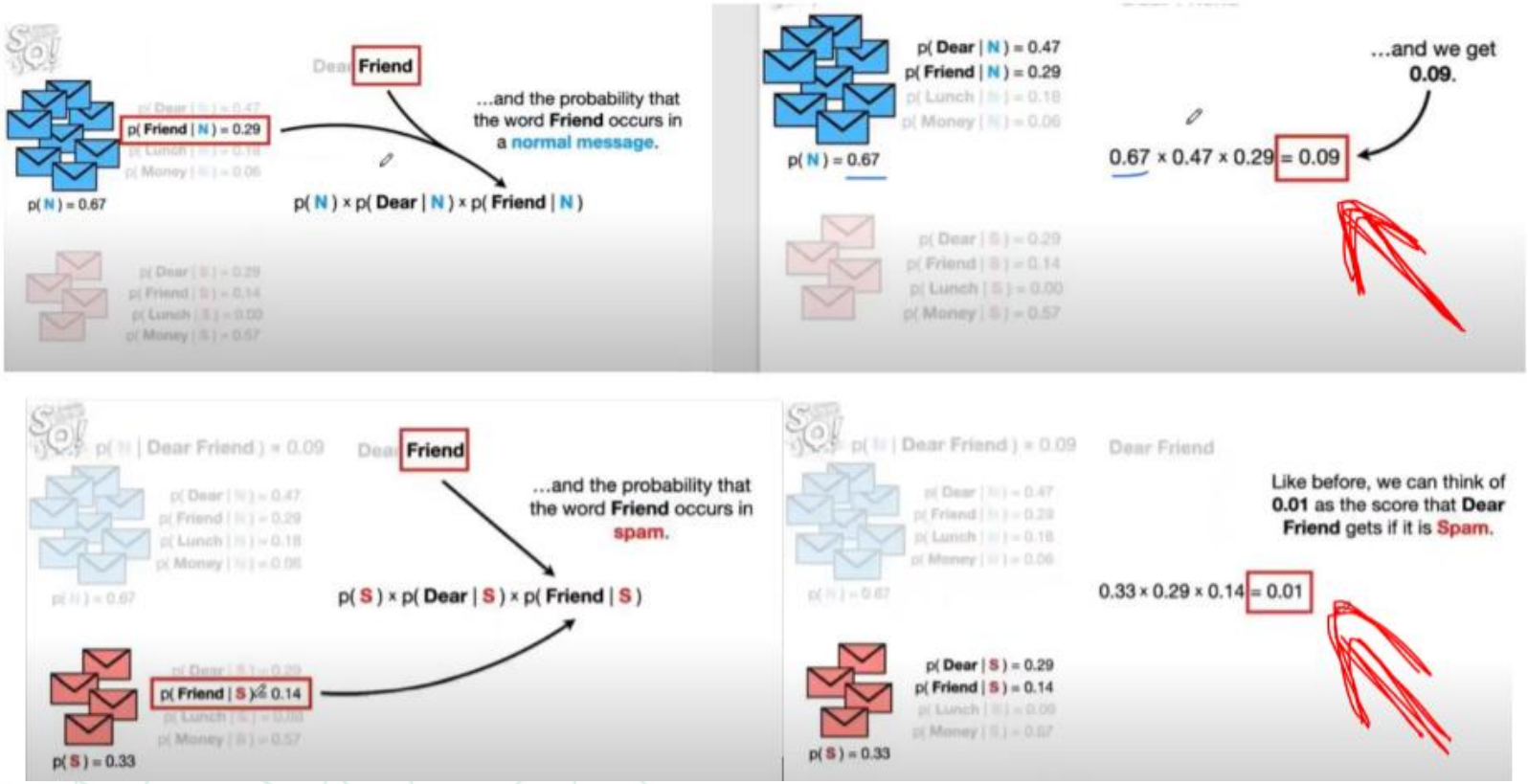
Naive Bayes

Classify New Messages Using Conditional Probabilities



Naive Bayes

Classify New Messages Using Conditional Probabilities



Naive Bayes

the process of combining the likelihood and evidence to reach the final classification

- We use the evidence (word frequencies) and likelihood (calculated in previous steps) to compute the posterior probabilities.
- The highest posterior probability determines the final classification of the message.

Naive Bayes

Step-by-Step Naive Bayes Example

Let’s walk through the key concepts, and then we’ll implement a Naive Bayes classifier with an example.

Naive Bayes

1. **Bayes' Theorem Explained**

Assume we want to classify whether an email is spam based on certain features (like **presence** of **specific words**).

- **Prior Probability $P(C)$:** The probability that any given email is spam based on historical data.
- **Likelihood $P(X|C)$:** The probability of observing the features (specific words) in spam emails.
- **Posterior Probability $P(C|X)$:** The probability that an email is spam given that certain words appear in the email.

Naive Bayes

2. **Assumption of Feature Independence**

- Naive Bayes assumes each feature contributes independently to the probability of the class.
- In reality, some features may be correlated, but this assumption simplifies calculations and often yields surprisingly good results.

Naive Bayes

3. **Mathematical Representation**

- For multiple features X1,X2,...,Xn, the posterior probability is

$$P(C|X_1, X_2, ..., X_n) = \frac{P(C) \cdot P(X_1|C) \cdot P(X_2|C) \cdot ... \cdot P(X_n|C)}{P(X_1, X_2, ..., X_n)}$$

- Since $P(X1,X2,...,Xn)$ is constant across classes, we focus on maximizing the numerator

Naive Bayes

Gaussian Distribution Assumption

In **Gaussian Naive Bayes**, we assume the data follows a normal distribution. For each feature Xi, the likelihood is modeled as

$$P(X_i|C_k) = \frac{1}{\sqrt{2\pi\sigma_k^2}} \exp\left(\frac{-(X_i - \mu_k)^2}{2\sigma_k^2}\right)$$

Where :

- Xi is the feature value.
- μk is the mean of the feature for class Ck
- σk is the variance of the feature for class Ck
- This formula is applied to each feature independently.

Naive Bayes

Advantages and Disadvantages of Naive Bayes

Advantages of Naive Bayes

1. **Works Well on Small Datasets**: Naive Bayes can perform well even with small amounts of training data.

2. **Handles Irrelevant Features**: Since Naive Bayes assumes that features are independent, it can effectively ignore irrelevant features, meaning their presence doesn't negatively impact the model's performance.

Naive Bayes

Advantages and Disadvantages of Naive Bayes

Disadvantages of Naive Bayes

1. **Assumes Feature Independence**: In the real world, features are often correlated, which violates the independence assumption made by Naive Bayes. This can lead to suboptimal performance in situations where feature interactions are important.

2. **Not Generalized for Complex Data**: The model may fail to generalize well if the data structure is complex and the relationships between features are not truly independent.

Naive Bayes

Can Naive Bayes Be Used for Regression?

Primarily for Classification: Naive Bayes is typically used for classification tasks like text classification, spam filtering, and sentiment analysis, where the goal is to classify data into discrete categories.

Adaptations for Regression

- Some adaptations of Naive Bayes, such as **Gaussian Naive Bayes**, can be used for regression tasks.
- **Gaussian Naive Bayes for Regression**: In this case, the algorithm predicts the **mean of the distribution** given the input features, assuming that the target variable follows a Gaussian (normal) distribution.